



PRESENTATION SCRIPT & Q&A PLAYBOOK

PSS06 — Automated Press Title Admissibility Clearance · PRGI TitleGuard

Team 404 Found (ID 26) · Divvy · Pruthviraj · Jai · Suhani · Gurpreet · Darsh
15 minutes total — 10 minutes presentation, 5 minutes Q&A

The 15 minutes, mapped

DIVVYE Open & frame 0:00–1:30	PRUTHVIRAJ Rules & proof 1:30–3:00	JAI 4-D matching 3:00–4:40	SUHANI Agents & RAG 4:40–6:10	GURPREET Live system 6:10–7:40	DARSH Officer side 7:40–9:10	DIVVYE Close 9:10–10:00	Q & A All six on their feet 10:00–15:00
--	---	---	--	---	---	--------------------------------------	--

Clock	Speaker	What they cover	Deck slide
0:00–1:30	Divvy	Problem, the 25–30 day gate, and the one design insight: similarity and statute are two different problems	1 → 2
1:30–3:00	Pruthviraj	The deterministic rule engine, verbatim clause citations, published coverage gaps	2
3:00–4:40	Jai	Four-dimensional similarity + the consonant-skeleton trick + fusion. Live demo runs here.	2 → 3
4:40–6:10	Suhani	RAG explainer with the anti-hallucination guard; the 4-agent Title Studio and its retry loop	3
6:10–7:40	Gurpreet	Live/offline engine badge, tested fallback, real stage timings, one-command startup	3 → 4
7:40–9:10	Darsh	Officer docket, memorandum export, accessibility, end-to-end tests	4 → 5
9:10–10:00	Divvy	Measured numbers, the recommendation-not-decision close, references	5 → 6

Rules of the room

- **Hand off by name.** Every block ends with the next person's name. That is the cue — nobody waits to be looked at.
- **Never correct a teammate mid-sentence.** If something is wrong, the next speaker fixes it in one clause and moves on.
- **Route questions you do not own.** “That’s Jai’s area — Jai?” is a strong answer. Guessing in someone else’s domain is the only way to lose credibility fast.
- **Hard clock cues.** If Suhani has not started by **4:50**, whoever is speaking cuts to their last sentence. At **9:10** Divvy takes over regardless of where the group is.
- **Only one person drives the laptop** — Gurpreet. Speakers point, they do not reach for the trackpad.
- **Every highlighted number below must be replaced with the real measured value before you rehearse.** Do not say a number you have not personally seen.

Read this before you rehearse. The three-day plan records that the backend, rule engine, LangGraph flow, RAG explainer and test suite were all still to be built when the plan was written. Whatever is genuinely finished on the day is what you claim — nothing more. Where a stage is still on its stub flag, the honest answer (“that stage is stubbed, here is what is real”) costs you far less than a claim a judge can disprove in ten seconds. The Q&A answers below are written so they hold up either way.

The script — 10 minutes, word for word

Delivered at a normal pace this runs to time. **Do not memorise it verbatim** — learn the shape and the three or four sentences that carry each block, then say it in your own words. The word counts are there so you can check your pace against a stopwatch: roughly 150 words a minute.

0:00 – 1:30

DIVVYE

Integration lead · contracts, FastAPI orchestrator, semantic scoring & fusion

~210 words

SLIDE 1 ON SCREEN. STAND CENTRE. DO NOT START UNTIL THE ROOM IS QUIET.

Good morning. We're Team 404 Found, and we're here with PSS06 — automated press title admissibility clearance for the Press Registrar General of India.

Here is the situation today. Under the Press and Registration of Periodicals Act, 2023, no periodical in India can publish until its title has been cleared against every title already on the register. That register holds 82,713 entries. The check is manual. It takes twenty-five to thirty days. And if you are rejected, you are not told why — you pay roughly a thousand rupees again and start over.

ADVANCE TO SLIDE 2.

Two completely different things can sink a title, and this is the insight the entire system is built on. One — it is confusingly similar to something already registered. Two — it breaks a statutory naming rule, regardless of similarity. Those are structurally different problems. The obvious move is one classifier for both. We deliberately built two separate mechanisms — a four-dimensional similarity engine, and a deterministic rule engine — because only one of them is allowed to guess.

I own the contracts and the FastAPI orchestrator that ties all of this into a single verify call. Pruthviraj will start with the half that is *not* allowed to guess.

1:30 – 3:00

PRUTHVIRAJ

Rules corpus · deterministic rule engine · evaluation data

~215 words

I own the rules — which means I own whether you can trust anything else on this screen.

The PRGI Guidelines for Admissibility of Titles is a real document. I went through it one sentence at a time — every “shall not”, every “is not admissible” — and turned each one into a single machine-checkable rule with the clause attached *verbatim*. Not a paraphrase. The actual sentence. That is rules extracted, implemented and tested.

Two things I want to be explicit about, because they are the difference between a demo and a product.

First — where I could not find the clause, the rule is flagged as unverified and appears in our coverage report as a gap. We publish what the system *cannot* check. An invented clause number would discredit every other number we show you.

Second — the rule engine is plain Python. No language model decides admissibility. Same title, same rule, same answer, every time. That is what auditability means when a rejection is a legal act.

There is exactly one rule that needs a model — catching a banned meaning hidden inside a translated title. That one returns an advisory flag and requires human confirmation. It can never auto-fail a title.

Jai owns the half that *is* allowed to be fuzzy.

GURPREET DRIVES. HAVE THE FOUR DEMO TITLES PRE-TYPED IN A SCRATCH NOTE, NOT TYPED LIVE.

Similarity is where the manual process actually fails, because a title does not have to be identical to be confusing. We score every candidate on four independent dimensions.

Lexical — RapidFuzz, and specifically token-sort ratio, which sorts the words before it compares them. That is what catches “Times India” against the registered “India Times”. [\[DEMO 1 — Times India\]](#)

Phonetic — Double Metaphone. But Double Metaphone was designed for English surnames and it is weak on transliterated Indian names, so we added a second signal: the consonant skeleton. Strip the vowels. *Jagran* becomes J-G-R-N. *Jaagran* becomes J-G-R-N. Same code. Indian romanisation varies almost entirely in the vowels, so that one cheap rule catches most of the real phonetic collisions Metaphone misses. [\[DEMO 2 — Jaagran\]](#)

Semantic — multilingual embeddings over pgvector. “Dainik Samachar” and “Daily News” mean the same thing in two languages and land in the same vector neighbourhood. Nothing about the characters is similar. [\[DEMO 3 — Dainik Samachar\]](#)

Core-root — strip the media filler words: *The, Daily, Weekly, Express, News*. “The Vidarbha Daily Express” reduces to *Vidarbha*, and collides with the registered *Vidarbha Patrika*. [\[DEMO 4 — The Vidarbha Daily Express\]](#)

Those four are merged with Reciprocal Rank Fusion, which combines ranked lists by *position* rather than score — because a trigram score and a cosine similarity are not on the same scale and cannot be averaged.

And one rule overrides the blend: a perfect clash on any single dimension is disqualifying on its own. A title that *sounds* exactly like an existing one is a conflict even if it is spelt completely differently. Suhani.

Catching a bad title is half a product. Telling the applicant what to do next is the other half. Two pieces.

The explanation. When a rule fires, we do not ask a model “why was this rejected”. We retrieve the actual guideline passage from the database first — vector search over the chunked guideline text — and only then ask the model to phrase *that retrieved text* in two plain sentences. Retrieve, then generate.

And there is a guard. If the model’s output cites a clause that was not in what we retrieved, we throw the output away and quote the clause verbatim instead. That is how we can say we did not hallucinate, rather than hope.

The Agentic Title Studio. Four agents in a LangGraph graph — Interviewer, Generator, Verifier, Ranker. The Interviewer takes the publication’s genre, state and language and writes a brief. The Generator proposes fifteen to twenty candidates. The Verifier sends all of them back through the same five-stage pipeline you just watched — not a lighter check, the real one. And if fewer than five survive, it loops back to the Generator with the rejection reasons attached, up to three times.

[DEMO 5 — RUN THE STUDIO. POINT AT THE RETRY WHEN IT FIRES. THAT MOMENT IS THE MOST PERSUASIVE THING IN THE DEMO.]

So every alternative we put in front of an applicant has already passed the exact test that failed their original. We never show an unverified suggestion. Gurpreet.

I connect that engine to the interface, and I own what happens when it breaks.

Every result carries a badge saying which engine produced it — **LIVE**, against the full 82,713-title registry, or **OFFLINE**, against a local sample. The two are not guaranteed to agree, so the user is told which one answered rather than left to assume.

The live call aborts after two and a half seconds. If the backend hangs we do not spin forever — we fall back to the offline engine and say so on screen.

[DEMO 6 — KILL THE BACKEND, RE-RUN THE SAME TITLE, LET THE BADGE FLIP. DO THIS DELIBERATELY AND SLOWLY.]

That is not a slide claim. We killed the backend on purpose and ran the whole demo through the fallback, twice, on two different days. An untested fallback is not a fallback.

You are also not watching a generic spinner. The pipeline shows which of the five stages is running, and when the response lands it snaps to the real measured per-stage timings — **normalize __ms · shortlist __ms · score __ms**. Measured, not animated.

And the whole thing is one command. `docker compose up` brings up PostgreSQL with pgvector, the backend and the frontend. There is also a thirty-second path without Docker, because a laptop building a PyTorch image at nine in the morning is how demos die. Darsh.

Everything so far serves the applicant. This half serves the officer who has to sign the decision.

The Review Docket sorts **amber first** — the borderline cases at the top, by descending risk. That is the docket's entire purpose: the officer's attention goes to the genuinely difficult cases, not the obvious ones.

Opening a case shows *exactly* the same evidence the applicant saw — all four sub-scores, the top five clashing titles with their state and language, and every rule triggered with its verbatim clause. Same components, not a second display, so the officer and the applicant can never be looking at different facts.

Two actions: *Endorse & Approve*, or *Issue Rejection Order*. Each produces a recorded artifact with a reference token and a timestamp. The memo itself is drafted by the system but fully editable, and requires an explicit officer action before it counts — it is a drafting aid, never an automated decision.

[DEMO 7 — OPEN A CASE, ENDORSE IT, THEN EXPORT THE MEMORANDUM.]

One click exports the whole thing as an official memorandum — title, verdict, all four scores, the clash table, every clause — to clipboard or to PDF.

On accessibility: no verdict is ever signalled by colour alone. Every colour is paired with a text label, and the verdict region announces itself to a screen reader when it arrives.

And we have  end-to-end browser tests — one per benchmark scenario, plus one that runs with the backend deliberately stopped. Divvy.

SLIDE 5. SLOW DOWN. THESE ARE THE SENTENCES THEY WILL REMEMBER.

To close with numbers rather than adjectives.

82,713 titles indexed. Shortlist recall at two hundred candidates: %, measured on a -pair evaluation set we built ourselves. Median end-to-end latency: ms. statutory rules implemented and tested, with our coverage gaps published rather than hidden.

Twenty-five to thirty days becomes under two seconds — and every verdict cites the clause it rests on.

One thing we want to be completely clear about. This system **recommends**. It never decides. The PRGI officer holds the legal authority, exactly as the Act requires. What we have removed is the waiting and the guessing — not the judgement.

We're Team 404 Found. Happy to take your questions.

Q & A — 5 minutes

Expect six to ten questions, not thirty. **Answer in three sentences and stop.** A long answer eats a question someone else could have answered better. If you do not know: "We haven't measured that — here is what we do know" is a respected answer; inventing a number is not recoverable.

DIVVYE — architecture, backend, integration, performance

Q Is this actually working, or is it a mockup? HIGH RISK

A Working. What you just saw came over HTTP from a FastAPI service querying PostgreSQL — the badge on the result says which engine answered, and I can show you the network tab. The offline engine is a genuine fallback against a smaller sample, not a fake.

If any pipeline stage is still on its stub flag on the day, name that stage explicitly and say what is real. Do not round up.

Q Six people, three days. How did you not step on each other?

A Contracts frozen at noon on day one — every request and response shape defined once, in Pydantic and in Zod, plus seven hand-written fixture files. After that nobody waited for anybody: Suhani's agents ran against a fixture client, Darsh's docket against fixture cases. Swapping a fixture for a real HTTP call is one line. On top of that, strict folder ownership — no two people edit the same directory.

Q How do you actually hit under two seconds?

A Four things. The embedding model loads once at startup, not per request — that alone is the difference between thirty seconds and thirty milliseconds. The four retrievers run concurrently. The two hundred shortlisted candidates are fetched in one query, not two hundred. And there is an in-memory cache on the normalised title.

Q Why merge with Reciprocal Rank Fusion instead of averaging the scores?

A Because a trigram similarity and a cosine similarity are not on the same scale, so averaging them is meaningless. RRF only looks at position — first place in any list is worth one over sixty-one, tenth is worth one over seventy. A title near the top of two different lists beats one that is first in only one, and it needs no calibration between scorers that were never comparable.

Q Why PostgreSQL and not a real vector database?

A 82,713 vectors is small — Pinecone and Milvus are built for hundreds of millions. Adding one would mean a second service, a second network hop per query, and keeping relational metadata and vectors in sync across two systems, for no gain at this size. With pgvector, one query filters on state and language and ranks by vector distance at the same time. If the register grows by two orders of magnitude that gets revisited — it is documented as a future migration, not an oversight.

Q Show me the guideline clause behind that rejection. Where exactly does it come from? **HIGH RISK**

A [Open the rule and read the clause field aloud.] That sentence is copied verbatim from section **1.1** of the Guidelines for Admissibility of Titles. Every rule row carries the sentence and the section it came from, and the rule code reads its citation out of that file at import time — so the code and the citation physically cannot drift apart. And here is the coverage report: every restriction we found in the document, and whether we implemented it.

Q What happens when you could not find the clause?

A The rule is flagged unverified and shows up as a gap in the coverage report. We publish what the system cannot check. A missing rule is a documented limitation; an invented clause number would discredit every citation in the system, so we never write one.

Q Your keyword lists will over-fire. Won't "News" flag half the register as commercial?

A That is exactly the failure we tested for. A single common word never triggers a rule on its own — we require a phrase or a second signal. We ran the whole rule set against fifty known-good registered titles and counted the false positives.

Q Your dataset is entirely Latin script. How can you claim multi-script support? **HIGH RISK**

A Correct, and we say so on the slide. The 82,713 real records are Latin-script only — that is a property of the source data, not a design choice. To demonstrate the cross-script path we generated a small native-script set with the indic-transliteration library, flagged in the database as synthetic. It is a demonstration set, back-transliteration is lossy, and it is labelled that way everywhere. The pipeline handles the script; we simply do not have real native-script registry data yet.

Q What is your accuracy? **HIGH RISK**

A We report shortlist recall at two hundred — **95%** on a **100**-pair evaluation set built from real near-duplicates in the register, the six benchmark scenarios, and hand-made reorderings, misspellings and translations. We deliberately do *not* quote a precision figure, because there is no ground-truth set of historical PRGI rejections to measure against. That is listed as an open risk, and it is exactly why a human stays in the loop on amber cases.

Q Why four dimensions? Isn't one good model enough?

A Because they fail in different places. Lexical cannot see that *Dainik Samachar* means *Daily News*. Semantic cannot see that *Jaagran* sounds like *Jagran*. Core-root catches *Vidarbha* buried inside *The Vidarbha Daily Express*, which nothing else does cleanly. Each is cheap; together they cover the failure modes the manual process actually misses.

Q Double Metaphone is English-centric. Doesn't it break on Indian names?

A It does — which is why we added the consonant skeleton alongside it. Strip the vowels and compare what is left: *Jagran*, *Jaagran* and *Jagaran* all reduce to J-G-R-N. Indian romanisation varies almost entirely in the vowels, so that one rule catches most of what Metaphone misses. We take the higher of the two signals.

Q Where do the weights come from? Aren't they arbitrary?

A They live in a config file loaded at startup, never hardcoded, because the spec requires them tunable. They were tuned against the evaluation set — we tried several combinations and kept the one with the best recall, and the committed file records the number it scored.

Q Scoring two hundred candidates on four dimensions — isn't that slow?

A Under three hundred milliseconds for all four. Nothing loops in Python over candidates — the lexical scorer runs vectorised in C++ against all two hundred at once, and the semantic scorer encodes the whole batch in a single call.

Q What is BM25 doing that trigram search isn't?

A Trigram matching treats every character sequence as equally informative. BM25 weights by rarity — matching "Vidarbha", which appears in about forty of 82,713 titles, counts enormously more than matching "News", which appears in thousands. That distinction is precisely what this problem needs.

Q How do you know the explanation isn't hallucinated? **HIGH RISK**

A Two things. Order — we retrieve the guideline passage *before* the model is called, and the prompt instructs it to use only that text. And a guard — we check that the output cites a clause that was actually in what we retrieved. If it does not, we discard the output and quote the clause verbatim. So the worst case is a blunter explanation, never a fabricated one.

Q Why LangGraph rather than CrewAI or AutoGen?

A Because this is not an open-ended agent conversation — it is a fixed four-step sequence with exactly one bounded retry. LangGraph expresses that as explicit nodes and edges with inspectable state at every step. The autonomous frameworks are built for looser collaboration, which here makes behaviour harder to constrain and audit — a disadvantage when the output has to be traceable.

Q Are those generated titles actually verified, or just generated?

A Actually verified, through the same five-stage pipeline, in one batch call — not a lighter check. If a generated title fails, it never reaches the applicant. That is why the Verifier is a node *inside* the graph rather than a step after it.

Q What if the loop never finds five good titles? Or the API dies mid-demo?

A The loop is bounded at three attempts, then returns whatever survived with an honest count — there is no unbounded loop anywhere in the graph. For the API: there is a deterministic template generator built on day one, and responses are cached on disk keyed by prompt hash, so a repeated run is instant and needs no network.

Q Isn't suggesting titles overstepping? That's the applicant's creative decision.

A It is optional, and it is a starting point rather than a submission — the applicant edits them or ignores them entirely. The value is that the ones we show have already cleared the check that failed their original, so they are not walking into a second rejection blind.

Q What happens if your backend goes down mid-demo? **HIGH RISK**

A [Do it, live.] The client aborts after two and a half seconds, falls back to the offline engine, and the badge changes to say so. We tested this by deliberately killing the backend and running the whole demo through the fallback, on two separate days. An untested fallback is not a fallback.

Q Then is the offline result trustworthy?

A It runs against a local sample, not the full register, and the badge says exactly that. The two engines are not guaranteed to agree — which is precisely why we surface which one answered instead of hiding it.

Q Those stage timings — real, or animation?

A The indicator advances optimistically while the request is in flight, then snaps to the real measured per-stage timings that come back in the response. The numbers you see are measured.

Q Your document says Next.js and this is React with Vite. Why the mismatch?

A The document was out of date and we chose to correct the document rather than the codebase. There is no server-side rendering requirement here, and migrating a working interface three days before a demo is how you lose a demo. It is a recorded decision, not an accident.

Q Could we run this ourselves?

A One command — `docker compose up` brings up the database, backend and frontend, with an example environment file listing every variable. There is also a no-Docker path that starts in about thirty seconds.

Q Does this replace the PRGI officer? **HIGH RISK**

A No — and the interface is deliberately built so it cannot read that way. The system produces a recommendation with evidence; the officer takes one of two explicit actions. Even the memo is editable and requires an officer action before it counts. The District Magistrate's endorsement step is untouched; it is out of scope by design.

Q Why does the officer see the same screen as the applicant?

A So they cannot be looking at different facts. We reuse the same components rather than building a second display, which means no version of the evidence exists on only one side. If the applicant is told their title clashes with *India Times* at ninety-four percent, that is the exact number in the officer's docket.

Q What is the audit trail on a decision?

A Each action produces a recorded artifact — a reference token and a timestamp appended to the case record. The exportable memorandum then contains the full basis for that decision: the title, the verdict, all four sub-scores, the clash table, every rule violated with its verbatim clause, and the plain-language summary.

Q Have you actually tested this, or does it only work in your demo?

A  end-to-end browser tests that drive a real browser — type the title, click verify, assert the verdict and the clash table — one per benchmark scenario. Plus one that runs with the backend deliberately stopped, so the fallback is covered by a test rather than by hope.

Q Is it usable by a colourblind officer?

A Every verdict colour is paired with an explicit text label — approved, manual review, rejected — never colour alone. The verdict region announces itself to a screen reader when it arrives, the docket is fully keyboard-navigable, and we specifically checked the amber-on-warm-background contrast, which is the combination that usually fails.

Everyone must be able to answer these five

These are asked of whoever is nearest, not of the person who owns the area. If you are asked one of these, answer it — do not deflect to a teammate.

1. **“Is it real?”** — point at the engine badge, offer the network tab. Name any stubbed stage rather than letting it be discovered.
2. **“Show me a citation.”** — open the actual clause. If it is one of the unverified ones, say so immediately and show the coverage report. That answer wins; a bluff loses everything else you said.
3. **“What's your accuracy?”** — recall at two hundred, with the evaluation set size. Never claim a precision figure. Name the missing ground truth as a known limitation.
4. **“Who is liable when it is wrong?”** — nobody, because it does not decide. Recommendation only; the officer decides; the Act is unchanged.
5. **“What have you *not* solved?”** — pick one and answer honestly: no ground-truth rejection data; the register being Latin-script only; “confusingly similar” being a legal judgement rather than a threshold. Judges reward a real answer here and punish “nothing”.

If something goes wrong

Situation	What you do
The live demo fails	Gurpreet says one sentence — “that's the fallback doing its job” — and plays the recorded capture. Do not debug on stage. Never apologise twice.
You are running long	Cut demo 3 and demo 4 first — the semantic and core-root cases. Keep Times India and Jaagran; they land fastest.

A judge interrupts mid-block	Answer in one sentence, then “I’ll come back to that in the Q&A” and resume. Do not let one question consume a speaking slot.
You are asked something outside your area	“That’s Pruthviraj’s — Pruthviraj?” Six specialists routing cleanly reads as a team; one person guessing reads as a solo project with passengers.
You genuinely do not know	“We haven’t measured that. What we do know is …” then give the nearest thing you <i>have</i> measured.
A teammate says something wrong	Let it stand. Fix it in one clause when your turn comes, without naming the error.